

Modul Praktikum Flutter Fundamental

Pertemuan 7: Mini Proyek Terpadu — TabungKu (To-Do List + Manajemen Keuangan)

Durasi	Level	Bentuk kerja	Prasyarat
2 sesi (sekitar 300 menit) atau 1 minggu	Menengah	Individu atau kelompok 2 orang	Pertemuan 1–6

1. Latar Belakang

Banyak mahasiswa kesulitan menabung karena pengeluaran kecil tidak tercatat. Padahal setiap kegiatan sehari-hari (kuliah, kerja kelompok, nongkrong, kerja paruh waktu) hampir selalu berkaitan dengan uang: ada yang mengeluarkan biaya, ada yang menghasilkan pemasukan. Pada mini proyek ini mahasiswa membangun **TabungKu**: aplikasi *to-do list* yang setiap kegiatannya dapat dicatat keuangannya, lalu diringkas menjadi laporan harian untuk melatih kebiasaan menabung.

2. Tujuan Pembelajaran

Setelah mengerjakan mini proyek ini, mahasiswa mampu:

1. Menganalisis kebutuhan sederhana dan menerjemahkannya menjadi aturan bisnis, rancangan data, dan daftar layar.
2. Mengintegrasikan materi Pertemuan 1–6 dalam satu aplikasi utuh: widget dan layout, form dan validasi, state, SQLite, *shared_preferences*, tema, *named routes*, *NavigationBar*, dan animasi.
3. Menulis query agregasi (COUNT, SUM, GROUP BY) untuk membuat laporan.
4. Menyusun kode ke dalam struktur folder yang rapi (model, data, state, widget, halaman).
5. Menguji aplikasi dengan skenario uji, mendokumentasikan, dan mendemonstrasikannya.

3. Deskripsi Aplikasi

Pengguna mendaftarkan kegiatan pada suatu tanggal. Untuk setiap kegiatan pengguna menandai **salah satu** dari tiga jenis: **Tidak melibatkan uang**, **Pemasukan** (dengan nominal), atau **Pengeluaran** (dengan nominal dan kategori). Kegiatan dapat dicentang selesai. Aplikasi menghitung laporan harian dan membandingkan tabungan hari itu dengan target menabung yang ditetapkan pengguna.

3.1 Aturan Bisnis

- Setiap kegiatan memiliki: nama, tanggal, status selesai, jenis keuangan, nominal, dan (khusus pengeluaran) kategori.
- **Tabungan harian = total pemasukan – total pengeluaran** dari semua kegiatan pada tanggal tersebut, baik yang sudah selesai maupun belum (keputusan desain; lihat pengembangan lanjutan untuk versi "hanya yang selesai").
- Nominal wajib berupa angka lebih dari 0 bila jenis keuangan adalah pemasukan atau pengeluaran.
- Kategori pengeluaran: Makan, Transport, Belajar, Hiburan, Lainnya.
- Target tabungan harian dapat diatur pengguna (bawaan Rp 20.000) dan tersimpan setelah aplikasi ditutup.
- Bila tabungan negatif, aplikasi menampilkan peringatan; bila mencapai target, aplikasi memberi apresiasi.

3.2 Contoh Skenario Satu Hari

Kegiatan	Jenis	Nominal	Kategori	Selesai
Kuliah pagi dan makan siang	Pengeluaran	Rp 15.000	Makan	Ya
Pulang naik angkot	Pengeluaran	Rp 10.000	Transport	Ya
Kerja freelance desain poster	Pemasukan	Rp 50.000	-	Belum
Belajar Flutter di perpustakaan	Tidak	-	-	Belum

Hasil laporan hari itu	Nilai
Jumlah kegiatan (selesai/total)	2/4
Total pemasukan	Rp 50.000
Total pengeluaran	Rp 25.000 (Makan Rp 15.000, Transport Rp 10.000)
Tabungan hari itu	Rp 25.000 (melebihi target Rp 20.000, jadi target tercapai)

4. Fitur Wajib

Kode	Fitur	Materi terkait
F1	Daftar kegiatan hari ini dengan centang selesai, ubah, dan hapus (dengan konfirmasi)	P2, P3, P5
F2	Form tambah/ubah kegiatan: nama, tanggal (date picker), jenis keuangan, nominal, kategori; semua tervalidasi	P3
F3	Penyimpanan data di SQLite (tabel kegiatan) sehingga tidak hilang saat aplikasi ditutup	P5
F4	Ringkasan hari ini: jumlah kegiatan, pemasukan, pengeluaran, tabungan	P4, P5
F5	Halaman Laporan: pilih tanggal, ringkasan, pengeluaran per kategori, dan 7 hari terakhir	P4, P5
F6	Target tabungan harian + indikator kemajuan beranimasi	P5, P6
F7	Pengaturan: mode gelap, warna tema, target; tersimpan	P5, P6
F8	Navigasi tiga tab (NavigationBar) dan halaman form lewat named route	P2, P6

5. Rancangan

5.1 Struktur Folder

lib/	
main.dart	tema + named routes
models/kegiatan.dart	Kegiatan, Tipe, Ringkasan
data/db_helper.dart	SQLite (CRUD + query laporan)
state/pengaturan.dart	tema, warna, target (shared_preferences)
utils/format.dart	format tanggal dan rupiah
widgets/ringkasan_card.dart	kartu ringkasan + progress target
widgets/kegiatan_tile.dart	satu baris kegiatan
pages/shell_page.dart	NavigationBar 3 tab
pages/beranda_page.dart	kegiatan hari ini
pages/form_kegiatan_page.dart	tambah/ubah kegiatan
pages/laporan_page.dart	laporan
pages/pengaturan_page.dart	pengaturan

5.2 Rancangan Tabel

Kolom	Tipe	Keterangan
id	INTEGER, PK, AUTOINCREMENT	Kunci utama
judul	TEXT, NOT NULL	Nama kegiatan
tanggal	TEXT, NOT NULL	Format yyyy-MM-dd agar mudah dibandingkan dan diurutkan
selesai	INTEGER, 0/1	SQLite tidak memiliki tipe boolean
tipe	TEXT	tanpa, pemasukan, atau pengeluaran
nominal	INTEGER	Rupiah bulat (tanpa desimal); 0 bila tanpa uang
kategori	TEXT	Diisi hanya untuk pengeluaran, selain itu '-'

5.3 Alur Data dan Halaman

Halaman	Isi utama	Sumber data
Hari Ini	Kartu ringkasan + daftar kegiatan hari ini + tombol tambah	DbHelper.pada(), DbHelper.ringkasan()
Form Kegiatan	Nama, tanggal, jenis keuangan, nominal, kategori	Menulis via DbHelper.tambah() / ubah()
Laporan	Pilih tanggal, ringkasan, per kategori, 7 hari terakhir	ringkasan(), pengeluaranPerKategori()
Pengaturan	Slider target, saklar gelap, pilihan warna	shared_preferences

Menyegarkan tampilan. Ada tiga tab yang menampilkan data yang sama, sehingga perubahan di satu tempat harus terlihat di tempat lain. Solusinya sederhana: berkas `data/db_helper.dart` memiliki variabel global `ValueNotifier<int> versiData` yang dinaikkan oleh `DbHelper` setiap kali ada tambah/ubah/hapus. Halaman yang menampilkan data mendengarkannya (`addListener()`) dan memuat ulang `Future`-nya. Jangan lupa `removeListener` di `dispose()`.

6. Panduan Pengerjaan Bertahap

Kerjakan berurutan; setiap tahap berakhir dengan *checkpoint* yang harus berjalan sebelum lanjut. Perkiraan waktu untuk pengerjaan individu.

Tahap	Kegiatan	Checkpoint	Waktu
1	Analisis dan rancangan: gambar sketsa 3 layar, tetapkan tabel dan aturan bisnis. Buat proyek dan pasang paket.	Sketsa dan skema tabel diperiksa asisten	30 mnt
2	Model (Kegiatan, Ringkasan), utilitas format, DbHelper (CRUD).	Data uji tersimpan dan terbaca (cek lewat <code>print</code> atau layar sementara)	40 mnt
3	Halaman Hari Ini: daftar kegiatan, centang selesai, hapus dengan konfirmasi.	Data dari SQLite tampil dan bertahan setelah restart	40 mnt
4	Form kegiatan dengan validasi; jenis keuangan menampilkan/menyembunyikan nominal dan kategori.	Tambah dan ubah berhasil; input salah ditolak	40 mnt

Tahap	Kegiatan	Checkpoint	Waktu
5	Query ringkasan dan kartu ringkasan; halaman Laporan (per kategori, 7 hari).	Angka laporan cocok dengan hitungan manual	50 mnt
6	Pengaturan, tema, target tabungan, progress beranimasi, NavigationBar.	Pengaturan bertahan setelah restart	40 mnt
7	Pengujian dengan skenario uji, perapihan kode, README, tangkapan layar, persiapan demo.	Semua skenario uji lulus	60 mnt

6.1 Petunjuk Kode Kunci

Tiga potongan berikut adalah bagian yang paling sering membuat mahasiswa tersendat. Sisanya dapat dibangun dengan mengacu pada modul Pertemuan 1–6.

(a) Model dengan konversi ke/dari Map. Perhatikan selesai disimpan sebagai 0/1 dan tipe sebagai teks:

```
class Kegiatan {
  // ... properti dan constructor ...

  Map<String, Object?> toMap() => {
    'id': id,
    'judul': judul,
    'tanggal': tanggal,
    'selesai': selesai ? 1 : 0,
    'tipe': tipe.name,
    'nominal': nominal,
    'kategori': kategori,
  };

  factory Kegiatan.fromMap(Map<String, Object?> m) => Kegiatan(
    id: m['id'] as int,
    judul: m['judul'] as String,
    tanggal: m['tanggal'] as String,
    selesai: (m['selesai'] as int) == 1,
    tipe: Tipe.values.byName(m['tipe'] as String),
    nominal: m['nominal'] as int,
    kategori: m['kategori'] as String,
  );
}
```

(b) Query laporan harian. Satu query menghasilkan seluruh angka ringkasan. CASE WHEN memilah pemasukan dan pengeluaran; COALESCE menggantikan NULL (saat tidak ada data) dengan 0:

```
static Future<Ringkasan> ringkasan(String tanggal) async {
  final db = await _database;
  final hasil = await db.rawQuery('''
    SELECT COUNT(*) AS jumlah,
           COALESCE(SUM(selesai), 0) AS selesai,
           COALESCE(SUM(CASE WHEN tipe = 'pemasukan'
                             THEN nominal END), 0) AS masuk,
           COALESCE(SUM(CASE WHEN tipe = 'pengeluaran'
                             THEN nominal END), 0) AS keluar

    FROM kegiatan
    WHERE tanggal = ?
  ''', [tanggal]);
  final baris = hasil.first;
  return Ringkasan(
    jumlahKegiatan: baris['jumlah'] as int,
    selesai: baris['selesai'] as int,
    pemasukan: baris['masuk'] as int,
    pengeluaran: baris['keluar'] as int,
  );
}
```

(c) Memuat ulang saat data berubah. Pola initState / dispose dengan listener:

```
class _BerandaPageState extends State<BerandaPage> {
  late Future<DataHari> _future;

  @override
  void initState() {
    super.initState();
    _future = _ambil();
    versiData.addListener(_muat);
  }

  @override
  void dispose() {
    versiData.removeListener(_muat);
    super.dispose();
  }

  void _muat() {
    setState(() {
      _future = _ambil(); // Future baru -> FutureBuilder memuat ulang
    });
  }

  // ... _ambil() dan build() ...
}
```

7. Skenario Uji

Isi kolom hasil pada laporan. Semua skenario harus lulus sebelum demo.

No	Langkah	Hasil yang diharapkan
1	Tambah kegiatan "Belajar" jenis Tidak	Muncul di daftar; jumlah kegiatan naik; pemasukan/pengeluaran tidak berubah
2	Tambah kegiatan jenis Pemasukan Rp 50.000	Pemasukan Rp 50.000; tabungan naik Rp 50.000; nominal hijau dengan tanda +
3	Tambah kegiatan jenis Pengeluaran Rp 15.000 kategori Makan	Pengeluaran naik; tabungan turun; kategori Makan muncul di Laporan
4	Simpan kegiatan bernama kosong / nominal kosong / nominal 0	Pesan kesalahan tampil dan data tidak tersimpan
5	Centang selesai dua kegiatan	Kartu ringkasan menunjukkan selesai/total yang benar; judul tercoret
6	Ubah nominal kegiatan yang sudah ada	Ringkasan dan laporan ikut berubah tanpa restart
7	Hapus kegiatan (pilih Batal, lalu Hapus)	Batal tidak menghapus; Hapus menghapus dan ringkasan berkurang
8	Tambah kegiatan bertanggal kemarin	Tidak muncul di Hari Ini; muncul di Laporan pada tanggal itu dan pada daftar 7 hari
9	Ubah target dan warna, aktifkan mode gelap, tutup aplikasi lalu buka lagi	Semua pengaturan dan data tetap
10	Buat pengeluaran lebih besar dari pemasukan	Tabungan negatif berwarna merah dan muncul pesan peringatan

8. Pengembangan Lanjutan (Pilih Minimal 2)

- **Rencana vs realisasi:** laporan hanya menghitung uang dari kegiatan yang sudah selesai, dan menampilkan selisih terhadap rencana.
- **Laporan bulanan:** ringkasan per bulan dan grafik batang sederhana (misalnya dengan `Container` atau `CustomPaint`).
- **Pencarian dan filter** kegiatan berdasarkan nama, jenis, atau kategori (`LIKE`, `WHERE`).
- **Kategori kustom:** tabel `kategori` baru (naikkan `version` database dan tulis `onUpgrade`).
- **Kutipan motivasi menabung** dari API publik memakai `http` dan `FutureBuilder` (Pertemuan 4).
- **Kegiatan berulang** (misalnya uang saku mingguan sebagai pemasukan tetap).
- **Ekspor laporan** sebagai teks yang dapat disalin ke papan klip (`Clipboard`).

9. Ketentuan Pengumpulan

- Repositori atau folder proyek (tanpa folder `build/`) yang dapat dijalankan dengan `flutter run`.
- **README** berisi: deskripsi, daftar fitur, cara menjalankan, struktur folder, dan fitur pengembangan yang dipilih.
- Tabel **hasil skenario uji** (lulus/tidak, disertai catatan bila gagal).
- Minimal 6 tangkapan layar: Hari Ini, Form (dengan pesan kesalahan), Laporan, Pengaturan, mode gelap, dan tabungan negatif.
- Demo 5 menit: alur menambah kegiatan pemasukan dan pengeluaran, lalu tunjukkan laporan berubah.

10. Rubrik Penilaian

Komponen	Kriteria	Bobot
Fungsionalitas	Fitur F1–F8 berfungsi; perhitungan laporan benar; skenario uji lulus	35%
Penerapan konsep	Tepat memakai SQLite, form validasi, state/refresh, tema, named routes, animasi	20%
Antarmuka dan pengalaman pengguna	Tampilan konsisten memakai tema, pesan jelas, kondisi kosong/galat ditangani	15%
Kualitas kode	Struktur folder, penamaan, tidak ada duplikasi berlebihan, controller di-dispose	10%
Pengujian dan dokumentasi	README, tabel uji, tangkapan layar	10%
Demo dan pengembangan lanjutan	Demo lancar, minimal 2 fitur lanjutan berfungsi	10%

11. Pertanyaan Refleksi

1. Mengapa tanggal disimpan sebagai teks `yyyy-MM-dd` dan bukan format lain seperti `dd/MM/yyyy`?
2. Mengapa laporan dihitung dengan query `SUM` di database, bukan dengan menjumlahkan di Dart? Kapan cara sebaliknya lebih tepat?
3. Bagaimana tiga tab dapat menampilkan data yang selalu sinkron? Apa risikonya bila `removeListener` lupa dipanggil?
4. Apa dampak keputusan "semua kegiatan dihitung, bukan hanya yang selesai" terhadap kebiasaan menabung pengguna?

5. Bagian mana dari aplikasi ini yang paling sulit, dan apa yang akan Anda ubah bila mengerjakannya ulang?

12. Troubleshooting Umum

Masalah	Solusi
Ringkasan tidak berubah setelah tambah/ubah	Pastikan operasi database menaikkan <code>versiData</code> dan halaman mendengarkannya
Error: type 'Null' is not a subtype of type 'int'	Hasil <code>SUM</code> bisa <code>NULL</code> bila tidak ada baris; bungkus dengan <code>COALESCE(..., 0)</code>
no such table / no such column	Skema berubah tanpa menaikkan <code>version</code> ; saat pengembangan hapus data aplikasi lalu jalankan ulang
Nominal masih terlihat setelah memilih "Tidak"	Tampilkan field nominal hanya bila jenis bukan <code>tanpa</code> dan simpan 0 pada kasus itu
<code>MissingPluginException</code>	Hentikan aplikasi sepenuhnya dan jalankan ulang setelah menambah paket
Layar tertutup keyboard di form	Pakai <code>ListView</code> sebagai induk form
<code>flutter test</code> gagal	<code>test/widget_test.dart</code> bawaan menguji aplikasi counter; hapus atau ganti dengan pengujian sendiri

13. Catatan untuk Pengajar

- Berkas terpisah **tabungku-contoh-mini-proyek.zip** berisi contoh solusi lengkap (folder `lib/` dan `README`). Sebaiknya **tidak dibagikan sebelum tenggat**, atau dibagikan setelah pengumpulan sebagai pembahasan.
- Cara menjalankan contoh solusi ada pada `README` di dalam zip. Contoh solusi telah diperiksa dengan `flutter analyze` (tanpa masalah), di-*build* menjadi APK, dan logika database-nya diuji otomatis dengan skenario uji 1–3, 5–8, dan 10 pada Flutter 3.47.5. Tetap jalankan sekali di perangkat sebelum dipakai di kelas.
- Contoh solusi dirancang agar mahasiswa yang telah menyelesaikan Pertemuan 1–6 dapat memahaminya: satu-satunya gagasan baru adalah `versiData` (pola *observer* sederhana) dan query agregasi.
- Variasi tema untuk mencegah kesamaan antarmahasiswa: Bekal Harian (uang jajan), Kas Kelompok, atau Agenda Usaha Kecil, dengan struktur data dan laporan yang sama.

14. Referensi

- Dokumentasi Flutter: docs.flutter.dev
- Paket `sqflite`: pub.dev/packages/sqflite
- Paket `shared_preferences`: pub.dev/packages/shared_preferences
- Modul Pertemuan 1–6 praktikum ini